

JavaScript Fundamentals JS DOM

Lecture 3.

1. Understand the basics of **JavaScript fundamentals**
2. Explain what the **DOM (Document Object Model)** is
3. Select and manipulate HTML elements using **JavaScript**
4. Add **dynamic behavior** to static HTML pages
5. Handle basic **user interactions** such as clicks and input events
6. Create simple **client-side interactivity**

Learning Outcome

By the end of this topic, students will be able to:

- ▶ Add **dynamic behavior** to static HTML pages
- ▶ Use **JavaScript** to respond to user actions
- ▶ Manipulate page content **without reloading** the browser
- ▶ Create **simple interactive features** on the client side
- ▶ This allows students to transform **static websites** into **interactive web pages**.

```
mirror_mod = modifier_ob.  
Set mirror object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
  
selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_obj  
data.objects[one.name].select  
  
print("please select exactly  
  
-- OPERATOR CLASSES ----  
  
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
  
context):  
context.active_object is not
```

Plan



JavaScript: Overview



ES6+ features:
let/const, arrow
functions, template
literals



DOM manipulation:
querySelector, events,
dynamic updates



Form validation and
input handling



Simple client-side
interactivity

JavaScript: Overview

JavaScript is a programming language used to make web pages **interactive**.

It allows websites to:

- ▶ Respond to user actions
- ▶ Update content dynamically
- ▶ Validate forms
- ▶ Control behavior in the browser

JavaScript works together with:

- ▶ **HTML** — structure
- ▶ **CSS** — appearance
- ▶ **JavaScript** — behavior



JavaScript is a programming language, mostly used to define the interactive behavior of web pages. It was invented by Brendan Eich.

JavaScript allows you to make web pages more interactive by accessing and modifying the contents and styling in a web page while it is being viewed in the browser.

JavaScript, along with HTML and CSS, forms the foundation of modern web browsers and the internet.

JavaScript is considered, unofficially, to be the most popular programming language in the world .

Numerous open-source mapping and data visualization libraries are written in JavaScript, including **Leaflet** and **MapLibre GL JS**, as well as **OpenLayers**, **D3**, and many others.

Many commercial services of web mapping also provide a JavaScript API for building web maps with their tools, such as **Google Maps JavaScript API**, **Mapbox GL JS** and **ArcGIS API for JavaScript**

Examples of what JavaScript can do

1. JavaScript Can Change HTML Content:

One of many JavaScript HTML methods is `getElementById()`.

The example "finds" an HTML element (with `id="demo"`), and changes the element content (`innerHTML`) to "Hello JavaScript":

```
document.getElementById("demo").innerHTML = "Hello JavaScript";
```

(The Element property `innerHTML` gets or sets the HTML markup contained within the element.)

2. JavaScript Can Change HTML Attribute Values.

3. JavaScript Can Change HTML Styles (CSS):

```
document.getElementById("demo").style.fontSize = "35px";
```

4. JavaScript Can Hide HTML Elements:

```
document.getElementById("demo").style.display = "none";
```

5. JavaScript Can Show HTML Elements (showing hidden HTML elements can also be done by changing the display style):

```
document.getElementById("demo").style.display = "block";
```

Adding JavaScript to Your Web Pages



There are typically three ways to add JavaScript to a web page:



Embedding the JavaScript code between a pair of `<script>` and `</script>` tag.



Creating an external JavaScript file with the `.js` extension and then load it within the page through the `src` attribute of the `<script>` tag.



Placing the JavaScript code directly inside an HTML tag using the special tag attributes such as `onclick`, `onmouseover`, `onkeypress`, `onload`, etc.

Embedding the JavaScript Code

You can embed the JavaScript code directly within your web pages by placing it between the `<script>` and `</script>` tags.

The `<script>` tag indicates the browser that the contained statements are to be interpreted as executable script and not HTML.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Embedding JavaScript</title>
</head>
<body>
  <script>
    var greet = "Hello World!";
    document.write(greet); // Prints: Hello World!
  </script>
</body>
</html>
```

External JavaScript

External file: myScript.js

```
function myFunction() {  
    document.getElementById("demo").innerHTML = "Paragraph changed.";  
}
```

External scripts are practical when the same code is used in many different web pages.

JavaScript files have the file extension **.js**.

To use an external script, put the name of the script file in the `src` (source) attribute of a `<script>` tag:

Example

```
<script src="myScript.js"></script>
```

- ▶ You can place an external script reference in `<head>` or `<body>` .
- ▶ The script will behave as if it was located exactly where the `<script>` tag is located.
- ▶ External scripts cannot contain `<script>` tags.

External JavaScript Advantages

Advantages of placing scripts in external files:

- ▶ It separates HTML and code
- ▶ It makes HTML and JavaScript easier to read and maintain
- ▶ Cached JavaScript files can speed up page loads

To add several script files to one page - use several script tags:

Example

```
<script src="myScript1.js"></script>  
<script src="myScript2.js"></script>
```

External References

An external script can be referenced in 3 different ways:

- ▶ With a full URL (a full web address)
- ▶ With a file path (like /js/)
- ▶ Without any path

This example uses a **full URL** to link to myScript.js:

Example

```
<script src="https://www.....com/js/myScript.js"></script>
```

Placing the JavaScript Code Inline

You can also place JavaScript code inline by inserting it directly inside the HTML tag using the special tag attributes such as *onclick*, *onmouseover*, *onkeypress*, *onload*, *etc.*

You should avoid placing large amount of JavaScript code inline as it clutters up your HTML with JavaScript and makes your JavaScript code difficult to maintain.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Inlining JavaScript</title>
</head>
<body>
  <button onclick="alert('Hello World!')">Click Me</button>
</body>
</html>
```

Positioning of Script inside HTML Document



The `<script>` element can be placed in the **<head>**, or **<body>** section of an HTML document.



Ideally, scripts should be placed at the end of the body section, just before the closing `</body>` tag, it will make your web pages load faster, since it prevents obstruction of initial page rendering.



Each **<script>** tag blocks the page rendering process until it has fully downloaded and executed the JavaScript code, so placing them in the head section (i.e. `<head>` element) of the document without any valid reason will significantly impact your website performance.

JavaScript in <head>

JavaScript function is placed in the <head> section of an HTML page.

The function is invoked (called) when a button is clicked:

Example

```
<!DOCTYPE html>
<html>
  <head>

    <script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
    </script>

  </head>

  <body><h2>Demo JavaScript in Head</h2>

  <p id="demo">A Paragraph</p>
  <button type="button" onclick="myFunction()">Try it</button>

  </body>
</html>
```

JavaScript in <body>

JavaScript function is placed in the <body> section of an HTML page.
The function is invoked (called) when a button is clicked:

Example

```
<!DOCTYPE html>

<html>
  <body>

<h2>Demo JavaScript in Body</h2>

<p id="demo">A Paragraph</p>

<button type="button" onclick="myFunction()">Try it</button>

    <script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
    </script>

  </body>
</html>
```

Difference Between Client-side and Server-side Scripting

Client-side scripting (JavaScript):

- ▶ Runs in the **web browser**
- ▶ Executes on the **user's computer**
- ▶ Provides **real-time interaction**
- ▶ Faster response

Server-side scripting (PHP, Python, Java, etc.):

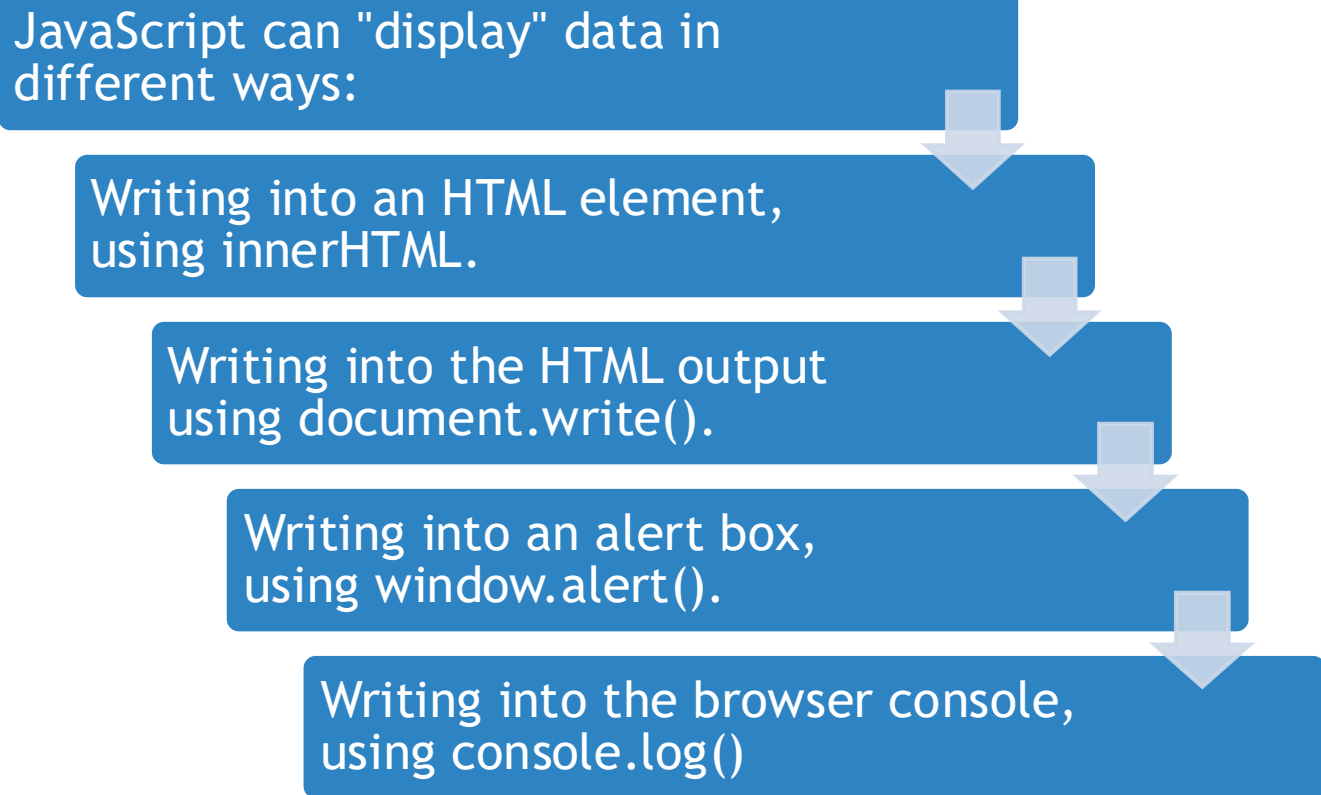
- ▶ Runs on the **web server**
- ▶ Sends results back as **HTML**
- ▶ Slower due to network communication

Advantages of Client-Side Scripting

- ▶ Instant user feedback (e.g. form validation)
- ▶ Reduced server load
- ▶ Saves network bandwidth
- ▶ Improves user experience

JavaScript Display Possibilities

JavaScript can "display" data in different ways:



```
graph TD; A[JavaScript can "display" data in different ways:] --> B[Writing into an HTML element, using innerHTML.]; B --> C[Writing into the HTML output using document.write().]; C --> D[Writing into an alert box, using window.alert().]; D --> E[Writing into the browser console, using console.log().];
```

Writing into an HTML element, using `innerHTML`.

Writing into the HTML output using `document.write()`.

Writing into an alert box, using `window.alert()`.

Writing into the browser console, using `console.log()`

Example: Simple Slideshow with Buttons (HTML + CSS + JavaScript)

HTML (structure)

```
<div class="slideshow">  
    
  
  <div class="controls">  
    <button id="prev">Previous</button>  
    <button id="next">Next</button>  
  </div>  
</div>
```

CSS (basic styling)

```
.slideshow {  
  width: 300px;  
  text-align: center;  
}
```

```
img {  
  width: 100%;  
  height: auto;  
}
```

```
.controls button {  
  margin: 10px;  
  padding: 8px 12px;  
}
```

JavaScript (DOM + interactivity)

How This Slideshow Works

- ▶ Images are stored in an **array**
- ▶ `currentIndex` tracks the current image
- ▶ Buttons listen for **click events**
- ▶ JavaScript changes the `src` attribute dynamically
- ▶ The page updates **without reloading**

```
const images = [
  "image1.jpg",
  "image2.jpg",
  "image3.jpg"
];

let currentIndex = 0;

const slide = document.querySelector("#slide");
const nextBtn = document.querySelector("#next");
const prevBtn = document.querySelector("#prev");

nextBtn.addEventListener("click", () => {
  currentIndex++;
  if (currentIndex >= images.length) {
    currentIndex = 0;
  }
  slide.src = images[currentIndex];
});

prevBtn.addEventListener("click", () => {
  currentIndex--;
  if (currentIndex < 0) {
    currentIndex = images.length - 1;
  }
  slide.src = images[currentIndex];
});
```

Using External JavaScript Files

JavaScript code can be placed in a **separate .js file**

The file is connected using the `<script>` tag and `src` attribute

Why Use External JavaScript?

- ▶ Reuse the same script on multiple pages
- ▶ Avoid repeating code
- ▶ Easier maintenance and cleaner HTML

HTML page



```
<script src="hello.js">
```



JavaScript file



Page behavior & interaction

```
<script src="js/hello.js"></script>
```

```
// A function to display a message
```

```
function sayHello() {  
    alert("Hello World!");  
}
```

```
// Call function on click of the button
```

```
document.getElementById("myBtn").onclick = sayHello;
```

```
!DOCTYPE html>
```

```
html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Including External JavaScript File</title>
```

```
</head>
```

```
<body>
```

```
    <button type="button" id="myBtn">Click Me</button>
```

```
    <script src="js/hello.js"></script>
```

```
</body>
```

```
</html>
```

ES6+ Features (Modern JavaScript)

ES6+ refers to modern versions of JavaScript that introduced cleaner and more powerful syntax.

Key ES6+ features include:

- ▶ let and const
- ▶ Arrow functions
- ▶ Template literals

These features make code:

- ▶ Easier to read
- ▶ Safer
- ▶ More maintainable

let and const

let and const are used to **declare variables**.

- ▶ let — value can be changed
- ▶ const — value cannot be reassigned

They replace the old var keyword and help avoid errors.

Benefits:

- ▶ Block-level scope
- ▶ Clearer code
- ▶ Better control over variables

Arrow Functions

Arrow functions are a **shorter way to write functions**.

They:

- ▶ Use the `=>` syntax
- ▶ Are more concise
- ▶ Improve readability

Arrow functions are commonly used for:

- ▶ Event handling
- ▶ Callbacks
- ▶ Simple logic

They make JavaScript code look cleaner and more modern.

Template Literals

Template literals use **backticks** instead of quotes.

They allow:

- ▶ Easy string interpolation
- ▶ Multi-line strings

This makes it easier to:

- ▶ Insert variables into text
- ▶ Build dynamic messages

Template literals improve readability and reduce string errors.

DOM Manipulation

The **DOM (Document Object Model)** represents the structure of a web page.

JavaScript can interact with the DOM to:

- ▶ Access elements
- ▶ Change content
- ▶ Update styles dynamically

DOM manipulation allows web pages to change **without reloading**.

How the Browser Creates the DOM

When a browser loads an HTML file, it does more than just display the page.

HTML file

↓

Browser loads HTML

↓

Browser reads tags, attributes, text

↓

Creates a DOM (object model)

↓

DOM is rendered on the screen (with CSS)

```
<!DOCTYPE html>
<html>
  <head>
    <title>A minimal web page</title>
  </head>
  <body>
    <h1>This is the main heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

The browser
receives an
HTML
document

Key Idea

The browser **builds a model** of the page in memory

This model stores:

- HTML elements
- Attributes
- Their order and hierarchy

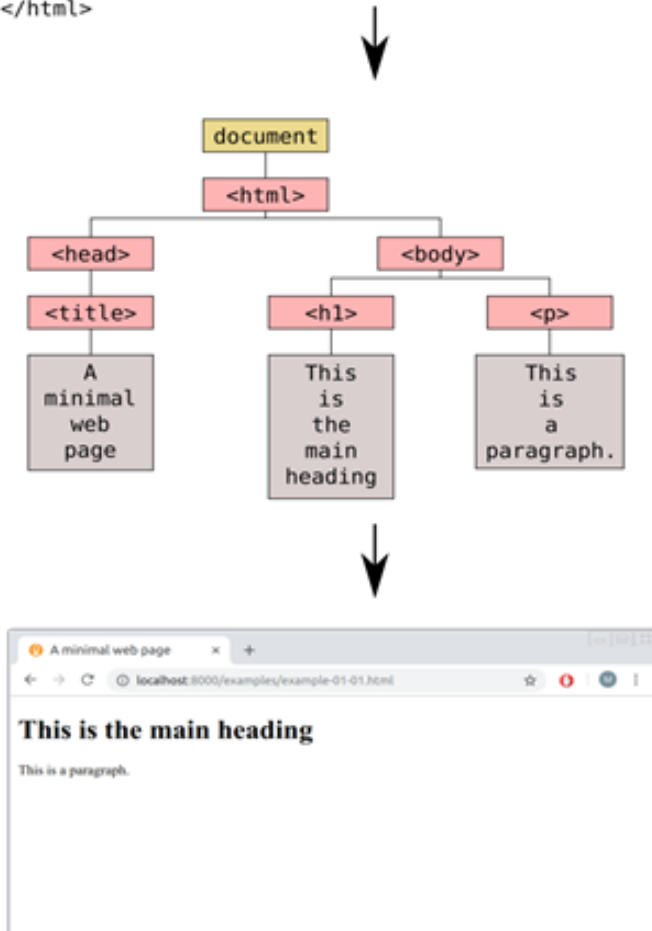
This model is called the **Document**

Object Model (DOM)

JavaScript uses the **DOM** to access and modify the page.

The browser
creates
a model
of the page

The browser
displays
the page
on screen



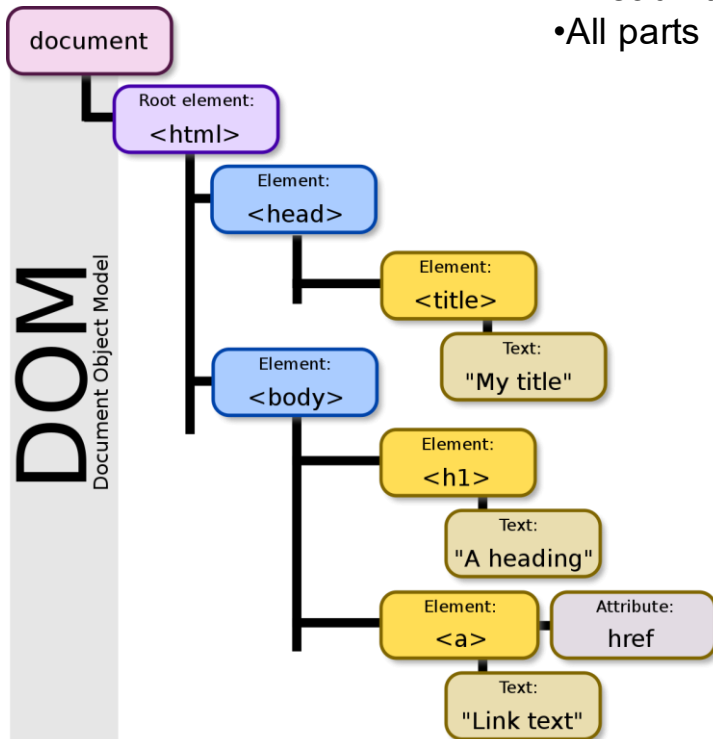
DOM as a Tree Structure

The **HTML element** is the **root (head)** of the DOM tree
It has two **child objects**:

- <head>
- <body>

```
html
├── head  (first child)
└── body  (last child)
```

- <head> and <body> are **child nodes** of html
- All parts of the document are called **nodes**



Why This Is Important

The DOM represents HTML as a family tree

JavaScript can:

- Access nodes
- Navigate the structure
- Add, change, or delete elements and attributes

Almost everything in an HTML document can be **manipulated using JavaScript through the DOM.**

DOM: Nodes and API (Summary)

- ▶ The **DOM** is standardized by **W3C** and **WHATWG**
- ▶ The DOM represents an HTML document as a **tree of nodes**

Types of DOM Nodes

- ▶ **Document node** – the entire document
- ▶ **Element nodes** – HTML elements
- ▶ **Attribute nodes** – element attributes
- ▶ **Text nodes** – text inside elements
- ▶ **Comment nodes** – HTML comments

DOM as an API

- ▶ The DOM is an **API** for JavaScript
- ▶ It connects JavaScript to HTML content
- ▶ Allows JavaScript to:
 - ▶ Navigate the page
 - ▶ Change content and structure

DOM Creation

- ▶ The browser creates the DOM when the page loads
- ▶ The DOM is accessed through the **document** object
- ▶ The **DOMContentLoaded** event signals that the DOM is ready

JavaScript and the DOM

The browser uses the **DOM tree** to represent all HTML elements, content, and styles

JavaScript runs in the browser and has **full access to the DOM**

JavaScript can:

- ▶ Read elements
- ▶ Add, change, or remove elements

When the DOM changes, the **browser updates the page automatically**

DOM and JavaScript Objects

- ▶ Each HTML element is treated as an **object**
- ▶ Objects have:
 - ▶ **Properties** — information about the object
 - ▶ **Methods** — actions that can be performed

The document Object

- ▶ document represents the entire web page
- ▶ It is the **entry point** for working with the DOM in JavaScript

TABLE 2-1

JavaScript Document Properties

Property	Description
<code>activeElement</code>	Returns the element that currently has the focus of the web page window
<code>anchors</code>	Returns a list of all the anchor elements on the web page
<code>body</code>	Sets or retrieves the body element of the web page
<code>cookie</code>	Returns all cookie names and values set in the web page
<code>characterSet</code>	Returns the character set defined for the web page
<code>documentElement</code>	Returns the DOM object for the html element of the web page
<code>documentMode</code>	Returns the mode used by the browser to display the web page
<code>domain</code>	Returns the domain name of the server used to send the document
<code>embeds</code>	Returns a list of all the embed elements in the web page
<code>forms</code>	Returns a list of all the form elements in the web page
<code>head</code>	Returns the head element for the web page
<code>images</code>	Returns a list of all the img elements in the web page
<code>lastModified</code>	Returns the time and date the web page was last modified
<code>links</code>	Returns a list of all the anchor and area elements in the web page
<code>title</code>	Sets or retrieves the title of the web page
<code>URL</code>	Returns the full URL for the web page

TABLE 2-2

JavaScript Document Methods

Method	Description
<code>createElement()</code>	Adds a new element object
<code>createTextNode()</code>	Adds a new text object
<code>getElementById(id)</code>	Returns an element object with the specified id value
<code>getElementsByName(class)</code>	Returns a list of elements with the specified class name
<code>getElementsByTagName(tag)</code>	Returns a list of elements of the specified element type
<code>hasFocus()</code>	Returns a true value if the web page has the window focus
<code>write(text)</code>	Sends the specified text to the web page
<code>writeln(text)</code>	Sends the specified text to the web page, followed by a new line character

JavaScript DOM object properties

Now that you have access to the objects contained in the web page, you can use JavaScript to manipulate them. Each DOM object contains one or more properties that define the actual object.

There are lots of object properties JavaScript uses with objects.

Property	Description
<code>attributes</code>	Returns a list of the object's attributes
<code>childElementCount</code>	Returns a list of the number of child objects the object has
<code>childNodes</code>	Returns a list of the object's child nodes, including text and comments
<code>children</code>	Returns a list of only the object's child element object nodes
<code>classList</code>	Returns a list of the class name attributes of an object
<code>className</code>	Sets or returns the value of a class attribute of an object
<code>firstChild</code>	Returns the first child object for the object
<code>id</code>	Sets or returns the id value of the object
<code>innerHTML</code>	Sets or returns the HTML content of the object
<code>lastChild</code>	Returns the last child object for the object
<code>nodeName</code>	Returns the name of the object
<code>nodeType</code>	Returns the element type of the object
<code>nodeValue</code>	Sets or returns the value for the object
<code>nextSibling</code>	Returns the next object at the same level in the tree as the object
<code>parentNode</code>	Returns the parent object for the object
<code>previousSibling</code>	Returns the previous object at the same level in the tree as the object
<code>style</code>	Sets or returns the value of the style property for the object

Using JavaScript to Modify the DOM

```
<h2 id="title">Welcome</h2>
<button id="btn">Change Text</button>
```

HTML

```
const title = document.querySelector("#title");
const button = document.querySelector("#btn");

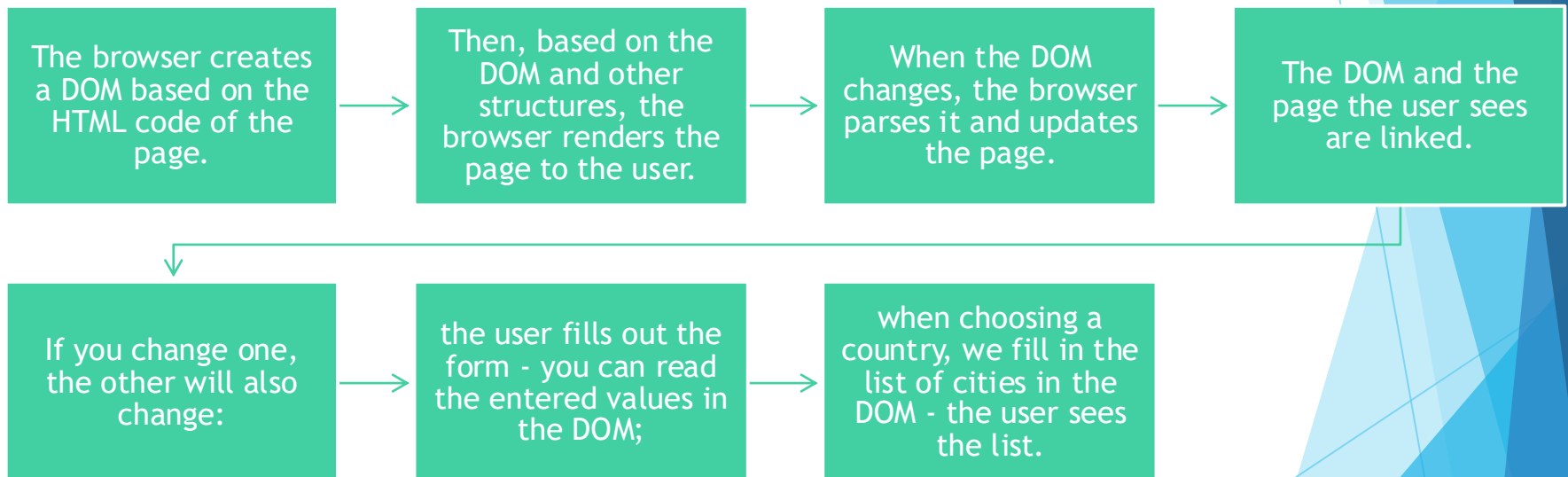
button.addEventListener("click", () => {
  title.textContent = "Hello, DOM!";
});
```

JavaScript

What Happens Here

1. The browser creates the **DOM tree**
2. JavaScript accesses elements using the **document** object
3. A **click event** is detected
4. JavaScript changes the content of an HTML element
5. The browser **updates the page instantly**

How it works



querySelector

querySelector is used to **select elements** from the page.

It works like CSS selectors:

- ▶ By tag
- ▶ By class
- ▶ By id

It allows JavaScript to target specific elements and work with them.

Events and Dynamic Updates

Events are user actions, such as:

- ▶ Clicks
- ▶ Typing
- ▶ Submitting forms

JavaScript listens for events and reacts to them.

Dynamic updates allow:

- ▶ Content to change instantly
- ▶ Better user experience
- ▶ Interactive interfaces

Form Validation and Input Handling

JavaScript is used to:

- ▶ Check user input
- ▶ Validate forms before submission
- ▶ Display error messages

This helps:

- ▶ Prevent incorrect data
- ▶ Improve usability
- ▶ Reduce server errors

Form validation makes websites more professional and user-friendly.

Simple Client-Side Interactivity

Client-side interactivity means **interaction happens in the browser**, without sending data to a server.

Examples:

- ▶ Showing or hiding content
- ▶ Updating text
- ▶ Reacting to button clicks

This makes websites:

- ▶ Faster
- ▶ More responsive
- ▶ More engaging for users

From Client-Side Interactivity to Client-Server Communication

So far, JavaScript interaction happens **inside the browser**

- ▶ Modern web applications also communicate with **servers**
- ▶ This communication allows:
 - ▶ loading external data
 - ▶ saving user input
 - ▶ working with databases

Modern web applications combine **DOM interactivity** and **server communication**

Client-Server Architecture

- ▶ The browser acts as a **client**
- ▶ The server provides data through an **API**
- ▶ Communication is done using **HTTP requests**
- ▶ The server sends responses back to the client
- ▶ **Optional diagram (simple):**

Browser (Client) → Request → Server

Browser ← Response ← Server

JSON as Data Exchange Format

Data exchanged between client and server is usually in **JSON**

JSON is:

- lightweight
- easy to read
- easy to parse in JavaScript
- Backend sends JSON
- Frontend receives JSON and converts it into objects

To request JSON data from a server, JavaScript needs a special mechanism.

Introducing the Fetch API - GET Request Example

Fetch API is a modern JavaScript interface

- Used to send HTTP requests from the browser
- Works asynchronously
- Built into modern browsers
- Commonly used with REST APIs
- Fetch connects JavaScript to backend APIs

```
fetch("https://api.example.com/data")  
  .then(response => response.json())  
  .then(data => {  
    console.log(data);  
  })  
  .catch(error => {  
    console.error(error);  
  });
```

- Sends a GET request to the server
- Receives a response
- Converts JSON into a JavaScript object
- Allows dynamic data usage in the page

Fetch API and Dynamic Pages

Fetch works together with:

- DOM manipulation
- Events
- User interaction

Data can be:

- loaded dynamically
- displayed without page reload

This enables **real-world web applications**

From Fetch API to Modern Frameworks

Fetch API is widely used in:

- ▶ Single Page Applications (SPA)
- ▶ React applications

In React:

- ▶ Fetch is often used inside components
- ▶ Fetch is a foundation for full-stack development

Fetch API is a key step between JavaScript fundamentals and React.

Summary

To summarize:

- ▶ ES6+ features modernize JavaScript
- ▶ `let`, `const`, arrow functions, and template literals improve code quality
- ▶ DOM manipulation allows dynamic pages
- ▶ JavaScript enables form validation and interactivity
- ▶ JavaScript is essential for building **modern, interactive web applications**
- ▶ JavaScript can communicate with servers using **Fetch API**

Questions:

1. What is JavaScript?

Answer:



JavaScript is a **lightweight, interpreted** programming language with object-oriented capabilities that allows you to build interactivity into otherwise static HTML pages.



The general-purpose core of the language has been embedded in Netscape, Internet Explorer, and other web browsers.

2. What are the data types supported by JavaScript?

The data types supported by JavaScript are:

Undefined

Null

Boolean

String

Symbol

Number

Object

3. What are the advantages of JavaScript?

Following are the advantages of using JavaScript :

Less server interaction – You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.



Immediate feedback to the visitors – They don't have to wait for a page reload to see if they have forgotten to enter something.



Increased interactivity – You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.



Richer interfaces – You can use JavaScript to include such items as drag-and-drop components and sliders to give a Rich Interface to your site visitors.

4. What is the difference between Attributes and Property?

Answer:

Attributes- provide more details on an element like id, type, value etc.

Property- is the value assigned to the attribute like type="text", value='Name' etc.

5. List out the different ways an HTML element can be accessed in a JavaScript code.

Here are the list of ways an HTML element can be accessed in a Javascript code:



- **getElementById('idname')**: Gets an element by its ID name



- **getElementsByClass('classname')**: Gets all the elements that have the given classname.



- **getElementsByTagName('tagname')**: Gets all the elements that have the given tag name.



- **querySelector()**: This function takes css style selector and returns the first selected element.

6. In how many ways a JavaScript code can be involved in an HTML file?

Answer :

There are 3 different ways in which a JavaScript code can be involved in an HTML file:

- ▶ **Inline**
- ▶ **Internal**
- ▶ **External**

An **inline** function is a JavaScript function, which is assigned to a variable created at runtime. You can differentiate between Inline Functions and Anonymous since an inline function is assigned to a variable and can be easily reused.

When you need a JavaScript for a function, you can either have the script **integrated** in the page you are working on, or you can have it placed in a **separate** file that you call, when needed.

This is the difference between an **internal** script and an **external** script.

7. What is the difference between null & undefined?

Answer



Undefined means a variable has been **declared** but has not yet been **assigned** a value.



On the other hand, null is an assignment value. It can be assigned to a variable as a representation of no value.



Also, undefined and null are two distinct types: undefined is a type itself (undefined) while null is an object.

Test

Quiz

1 Inside which HTML element do we put the JavaScript?

- A. `<javascript>`
- B. `<script>`
- C. `<scripting>`
- D. `<js>`

2 What is the correct JavaScript syntax to change the content of the HTML element below?

`<p id="demo">This is a demonstration.</p>`

- A. `document.getElementById("demo").innerHTML = "Hello World!";`
- B. `document.getElement("p").innerHTML = "Hello World!";`
- C. `#demo.innerHTML = "Hello World!";`
- D. `document.getElementByName("p").innerHTML = "Hello World! »`

3 Where is the correct place to insert a JavaScript?

- A. The `<head>` section
- B. Both the `<head>` section and the `<body>` section are correct
- C. The `<body>` section



Correct answers



1 Inside which HTML element do we put the JavaScript?

- A. `<javascript>`
- B. `<script>`
- C. `<scripting>`
- D. `<js>`

2 What is the correct JavaScript syntax to change the content of the HTML element below?

`<p id="demo">This is a demonstration.</p>`

- A. `document.getElementById("demo").innerHTML = "Hello World!"`
- B. `document.getElement("p").innerHTML = "Hello World!"`
- C. `#demo.innerHTML = "Hello World!"`
- D. `document.getElementByName("p").innerHTML = "Hello World! »`

3 Where is the correct place to insert a JavaScript?

- A. The `<head>` section
- B. Both the `<head>` section and the `<body>` section are correct
- C. The `<body>` section

4 What is the correct syntax for referring to an external script called "xxx.js"?

- A. `<script src="xxx.js">`
- B. `<script name="xxx.js">`
- C. `<script href="xxx.js">`

5 The external JavaScript file must contain the `<script>` tag.

- A. True
- B. False

6 How do you write "Hello World" in an alert box?

- A. `msgBox("Hello World");`
- B. `alertBox("Hello World");`
- C. `msg("Hello World");`
- D. `alert("Hello World");`



4 What is the correct syntax for referring to an external script called "xxx.js"?

- A. `<script src="xxx.js">`
- B. `<script name="xxx.js">`
- C. `<script href="xxx.js">`

5 The external JavaScript file must contain the `<script>` tag.

- A. True
- B. False

6 How do you write "Hello World" in an alert box?

- A. `msgBox("Hello World");`
- B. `alertBox("Hello World");`
- C. `msg("Hello World");`
- D. `alert("Hello World");`



7 How do you create a function in JavaScript?

- A. `function myFunction()`
- B. `function = myFunction()`
- C. `function:myFunction()`

8 How do you call a function named "myFunction"?

- A. `call function myFunction()`
- B. `myFunction()call`
- C. `myFunction()`

9 How to write an IF statement in JavaScript?

- A. `if i = 5`
- B. `if i == 5 then`
- C. `if (i == 5)`
- D. `if i = 5 then`



7 How do you create a function in JavaScript?

- A. `function myFunction()`
- B. `function = myFunction()`
- C. `function:myFunction()`

8 How do you call a function named "myFunction"?

- A. `call function myFunction()`
- B. `myFunction()call`
- C. `myFunction()`

9 How to write an IF statement in JavaScript?

- A. `if i = 5`
- B. `if i == 5 then`
- C. `if (i == 5)`
- D. `if i = 5 then`



10 How to write an IF statement for executing some code if "i" is NOT equal to 5?

- A. if (i <> 5)
- B. if i <> 5
- C. if (i != 5)
- D. if i != 5 then

11 How does a WHILE loop start?

- A. while (i <= 10)
- B. while (i <= 10; i++)
- C. while i = 1 to 10

12 How does a FOR loop start?

- A. for (i = 0; i <= 5)
- B. for i = 1 to 5
- C. for (i <= 5; i++)
- D. for (i = 0; i <= 5; i++)



10 How to write an IF statement for executing some code if "i" is NOT equal to 5?

- A. `if (i <> 5)`
- B. `if i <> 5`
- C. `if (i != 5)`
- D. `if i != 5 then`

11 How does a WHILE loop start?

- A. `while (i <= 10)`
- B. `while (i <= 10; i++)`
- C. `while i = 1 to 10`

12 How does a FOR loop start?

- A. `for (i = 0; i <= 5)`
- B. `for i = 1 to 5`
- C. `for (i <= 5; i++)`
- D. `for (i = 0; i <= 5; i++)`



13 How can you add a comment in a JavaScript?

- A. `<!--This is a comment-->`
- B. `//This is a comment`
- C. `'This is a comment`

14 How to insert a comment that has more than one line?

- A. `/*This comment has more than one line*/`
- B. `//This comment has more than one line//`
- C. `<!--This comment has more than one line-->`



13 How can you add a comment in a JavaScript?

- A. `<!--This is a comment-->`
- B. `//This is a comment`
- C. `'This is a comment`

14 How to insert a comment that has more than one line?

- A. `/*This comment has more than one line*/`
- B. `//This comment has more than one line//`
- C. `<!--This comment has more than one line-->`



15 What is the correct way to write a JavaScript array?

- A. `var colors = "red", "green", "blue" »`
- B. `var colors = 1 = ("red"), 2 = ("green"), 3 = ("blue")`
- C. `var colors = ["red", "green", "blue"]`
- D. `var colors = (1:"red", 2:"green", 3:"blue")`

16 How do you round the number 7.25, to the nearest integer?

- A. `Math.round(7.25)`
- B. `Math.rnd(7.25)`
- C. `rnd(7.25)`
- D. `round(7.25)`

17 How do you find the number with the highest value of x and y?

- A. `top(x, y)`
- B. `Math.max(x, y)`
- C. `Math.ceil(x, y)`
- D. `ceil(x, y)`



15 What is the correct way to write a JavaScript array?

- A. `var colors = "red", "green", "blue »`
- B. `var colors = 1 = ("red"), 2 = ("green"), 3 = ("blue")`
- C. `var colors = ["red", "green", "blue"]`
- D. `var colors = (1:"red", 2:"green", 3:"blue")`

16 How do you round the number 7.25, to the nearest integer?

- A. `Math.round(7.25)`
- B. `Math.rnd(7.25)`
- C. `rnd(7.25)`
- D. `round(7.25)`

17 How do you find the number with the highest value of x and y?

- A. `top(x, y)`
- B. `Math.max(x, y)`
- C. `Math.ceil(x, y)`
- D. `ceil(x, y)`



18 How can you detect the client's browser name?

- A. `client.navName`
- B. `browser.name`
- C. `navigator.appName`

19 JavaScript is the same as Java.

- A. False
- B. True

20 Is JavaScript case-sensitive?

- A. Yes
- B. No



18 How can you detect the client's browser name?

- ▶ `client.navName`
- ▶ `browser.name`
- ▶ `navigator.appName`

19 JavaScript is the same as Java.

- ▶ `False`
- ▶ `True`

20 Is JavaScript case-sensitive?

- ▶ `Yes`
- ▶ `No`

